SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1
Case Study

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS: **Total Marks: 25**

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

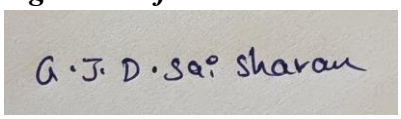
Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I G J D Sai Sharan / 192425152 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: CNN for Automated Medical Image Classification

1. Suitable CNN Architecture

A suitable architecture for classifying chest X-rays into **Normal** and **Disease-Affected** classes is:

Input Image → Convolution → ReLU → Batch Normalization → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Global Average Pooling → Fully Connected Layer → Sigmoid Output

Proposed Architecture

Layer	Purpose
Input	Resize X-ray to 224 × 224 × 1
Conv2D (32 filters, 3×3)	Detect edges and simple patterns
ReLU	Introduce non-linearity
Batch Normalization	Stabilize and accelerate training
Max Pooling (2×2)	Reduce spatial dimensions
Conv2D (64 filters, 3×3)	Learn textures and shapes
ReLU + Batch Normalization	Improve feature learning
Max Pooling	Reduce feature-map size
Conv2D (128 filters, 3×3)	Detect complex abnormal structures
ReLU + Batch Normalization	Improve representation
Max Pooling	Reduce computation
Global Average Pooling	Convert feature maps into compact representation
Dropout	Reduce overfitting
Fully Connected Layer	Perform classification
Sigmoid	Output probability of disease

For two classes, the final **sigmoid neuron** produces a value between 0 and 1. A value closer to 1 can represent disease-affected and a value closer to 0 can represent normal.

2. Motivation and Role of CNN Components

Motivation for Using CNN

CNNs are highly suitable for medical image classification because they can **automatically learn spatial and hierarchical features directly from images**. Unlike traditional methods, they do not require manual feature extraction.

For X-rays:

Edges → Textures → Shapes → Anatomical structures → Abnormal regions → Disease classification

Convolutional Layers

Convolutional layers apply learnable filters to the image and generate **feature maps**. Early layers detect simple features such as edges, while deeper layers detect complex structures and abnormalities.

Filters

Filters are small matrices such as **3×3** that slide across the image. Each filter learns to detect a particular visual pattern.

For example:

- Filter 1 → horizontal edges
- Filter 2 → vertical edges
- Deeper filters → textures and abnormal structures

Pooling Layers

Pooling reduces the spatial dimensions of feature maps while retaining important information.

Max pooling selects the maximum value from a local region.

Benefits:

- Reduces computation
- Reduces memory requirements
- Provides some translation tolerance
- Helps control overfitting

Fully Connected Layers

The extracted features are converted into a classification representation. The fully connected layer combines these learned features to distinguish **normal** and **disease-affected** X-rays.

3. Parameter Sharing

Parameter sharing is one of the major advantages of CNNs.

In a fully connected network, every input pixel may be connected to every neuron. For an image of **224 × 224 pixels**, there are:

224 × 224 = 50,176 input values

Connecting these to 1,000 neurons would require approximately:

50,176 × 1,000 = 50 million weights

In a CNN, a **3×3 filter** has only:

3 × 3 = 9 weights + 1 bias

The same filter is reused across the entire image.

Therefore, CNNs require **far fewer trainable parameters**, reducing memory requirements and computational cost while allowing the same feature to be detected at different locations.

4. Regularization Techniques

Medical image datasets can contain thousands of images but may still cause overfitting because the model can memorize training examples.

Dropout

Dropout randomly disables a percentage of neurons during training.

Example:

Dropout = 0.5

This prevents excessive dependence on particular neurons and improves generalization.

Data Augmentation

Training images can be randomly modified using:

- Small rotations
- Horizontal shifts
- Zooming
- Cropping
- Brightness variation
- Contrast adjustment

This increases the effective diversity of the training data.

Note: Augmentation must be medically appropriate; transformations that could change the clinical meaning of an X-ray should be avoided.

Batch Normalization

Batch normalization normalizes intermediate activations during training.

Benefits:

- More stable training
- Faster convergence
- Allows effective deeper networks
- Provides some regularization

Additional Measures

A suitable system should also use:

- Early stopping
- L2 regularization
- Proper train/validation/test splitting

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Relatively simple	Deep residual architecture
Depth	8 learnable layers	Much deeper, e.g., ResNet-50
Feature learning	Good	Excellent
Vanishing gradients	More difficult in deeper versions	Addressed using residual connections
Parameters	Relatively high for its age	More efficient for its depth
Training	Easier than very deep networks	Stable despite greater depth
Accuracy	Good baseline	Generally higher
Medical image suitability	Suitable for basic classification	Highly suitable for complex medical features

Recommendation: ResNet

ResNet is more appropriate for this application because chest X-rays contain complex and subtle visual patterns. Its **residual/skip connections** allow very deep networks to be trained effectively and reduce the vanishing-gradient problem.

A model such as **ResNet-50**, preferably using **transfer learning**, can learn rich hierarchical features while achieving strong classification performance.

Although ResNet requires more computation than AlexNet, the improved feature-learning capability and classification accuracy make it the better choice for a medical image classification system where **diagnostic performance is more important than minimum computational cost**.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

1. Suitable CNN Architecture

A CNN-based autonomous vehicle vision system can use the following architecture:

Camera Image → Input Layer → Convolution + ReLU → Pooling → Convolution + ReLU → Pooling → Deeper Convolution Layers → Feature Extraction → Fully Connected/Detection Head → Output

The system can be trained to recognize:

- Cars
- Pedestrians
- Traffic signs
- Bicycles
- Road obstacles

For an actual autonomous vehicle, a **CNN-based object detection architecture** is preferable to simple image classification because the system must identify **what the object is and where it is located**.

2. Role of CNN Layers and Progressive Feature Learning

Input Layer

Receives camera images, for example:

224 × 224 × 3

where 3 represents RGB channels.

Convolutional Layer

Applies filters to the input image to extract useful visual features.

Activation Function

ReLU introduces non-linearity:

ReLU(x) = max(0, x)

It allows the network to learn complex relationships.

Pooling Layer

Reduces feature-map dimensions while retaining important information.

Max Pooling is commonly used.

Fully Connected/Output Layer

Combines learned features and produces the final prediction.

For example:

Car → 0.92
Pedestrian → 0.05
Bicycle → 0.03

For object detection, the output also includes **bounding-box coordinates and confidence scores**.

Progressive Feature Learning

CNNs learn features hierarchically:

Early layers:
Edges → Lines → Corners

Middle layers:
Textures → Shapes → Wheels → Human body parts

Deep layers:
Cars → Pedestrians → Traffic signs → Complete objects

Thus, deeper layers provide increasingly meaningful representations.

3. Importance of Parameter Sharing

A CNN uses the same filter weights at different spatial locations.

For example, a filter trained to detect an edge can detect that edge whether it appears on the **left, center, or right** side of the image.

Therefore, parameter sharing:

- Greatly reduces the number of trainable parameters
- Reduces memory requirements
- Reduces computational complexity
- Allows the same feature to be detected at different positions
- Provides useful translation-related feature detection

This is especially important for autonomous vehicles because objects can appear at different positions in every camera frame.

4. Overfitting and Regularization

Overfitting occurs when the CNN performs very well on training images but poorly on unseen road scenes.

This may occur because of:

- Limited training diversity
- Similar training images
- Complex network architecture
- Noise in camera data

Recommended techniques

1. Data Augmentation

- Rotation
- Scaling
- Cropping
- Brightness changes
- Contrast changes
- Weather simulation

2. Dropout

Randomly disables neurons during training and reduces dependency on specific features.

3. Batch Normalization

Normalizes intermediate activations and improves training stability.

4. Early Stopping

Stops training when validation performance stops improving.

5. Transfer Learning

Starts with a model pretrained on a large image dataset and fine-tunes it using road-scene data.

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Conventional CNN	Residual CNN
Depth	8 learnable layers	Very deep, e.g., 18, 34, 50, 101+
Parameter efficiency	Lower	Better feature efficiency
Training difficulty	Relatively simple	More complex but stable
Vanishing gradient	More susceptible in deeper networks	Reduced using skip connections
Feature learning	Good	Excellent
Accuracy	Lower on complex modern tasks	Generally higher
Computational cost	Lower	Higher depending on variant
Complex scenes	Less effective	More effective
Transfer learning	Available	Excellent

Residual Connections in ResNet

ResNet introduces **skip connections**:

Input → Convolution Layers → Output + Input

Instead of forcing the network to learn the complete transformation, it learns a **residual mapping**.

This improves gradient flow and allows much deeper networks to be trained without severe vanishing-gradient problems.

6. Final Recommendation

ResNet is the better choice for the autonomous vehicle system.

Autonomous driving requires recognition of objects under **different lighting, distances, viewing angles, weather conditions, and complex backgrounds**. ResNet's deeper architecture can learn more powerful and discriminative features than AlexNet.

Comparison for the application

- **Accuracy:** ResNet is generally superior.
- **Feature learning:** ResNet learns complex hierarchical features effectively.
- **Training:** Skip connections reduce vanishing-gradient problems.
- **Computational cost:** AlexNet is lighter, but ResNet has more computational requirements.
- **Real-time requirement:** A suitable lightweight ResNet variant or optimized deployment can provide a good accuracy-speed trade-off.